Flash Attention EVU 优化路线分析

整理日期：2026-10-26；最近实测分析更新：2026-10-27。

本文记录 Flash Attention 核内 RVV + AME 实现与外挂 EVU 实现的差异，并按测试算法、硬件 RTL 两条路线列出后续优化方法。除明确标为已完成的改动外，以下内容均为优化建议，不表示已经实现或已经测得相应收益。源码行号以整理时的版本为准，后续修改可能使其偏移。

分析形状为 `q=128, kv=256, d=64`。历史 division-only cold/performance-only EVU 内核周期为 `1,970,504`；GatherImm 广播和操作数返回直通后的融合前内核为 `1,549,671`；软件 `row_fused_v1` 为 `1,257,794`；矩阵写合并与共享 tag 后为 `900,600`；bank 错位、读预取及单访存重叠后为 `764,160`。最新软件 `compact_batched_v2` 已实现并收到服务器日志，本轮结果分析按要求单独在终端给出。核内历史热跑为 `322,302`，冷/热及插桩条件不同，不能作为严格匹配条件下的最终加速比。

已落实的软件子集：row_fused_v1

下面的对比表和容量/流量数字保留为融合前的分析基线。共享内核现已把同一行的两个 scores chunk、max/alpha 更新与两次 softmax/量化合并，删除 alpha 数组及无效初始化，省去首次 output 清零/读取，并把最终归一化并入最后一次 PV 累加。首次 PV 仍与 +0 相加，以保持 signed-zero 和异常语义。默认形状按源码计数减少 2432 条 VL64 向量访存，即 608 KiB 逻辑读写；该数字不是实测总线流量或周期收益。

这一批没有实现完整的紧凑行状态、多行批处理、chunk 最大值复用或新的 RTL。当前广播状态数组为 128 KiB；若将同四份行状态压紧，数据本身仍为 2 KiB。新 breakdown 使用 `profile_schema=row_fused_v1`，为 23 个区间、9 个阶段，见 `FLASH_ATTENTION_BREAKDOWN.md`。

该软件批次的服务器日志已通过 difftest，`profile_errors=0`、`pending=0`，内核比融合前少 `291,877` 周期（18.83%，1.232 倍）。`row_softmax_quantize=611,682` 周期，较旧 max/alpha 和 softmax 合计少 27.15%；两类 output 阶段合计 `228,193` 周期，少 5.64%。QK/PV wait 合计 `382,991` 周期基本不变，占新内核 30.45%。整个仿真 host 时间为 `1,486,333 ms`（24 分 46.333 秒），不是单独内核 host 计时。这是 performance-only，`fp64_checked=0`，不能声称通过独立 FP64 精度验证。

已落实的 RTL 子集：矩阵写合并与共享 tag

以下 RTL 已实现并进行定向 Treadle 检查，其服务器工作负载验收见后文“矩阵写回批次服务器验收”：

- `EvuMatrixStoreBuffer.scala`：默认 8 拍输入 FIFO、16 个按 64B 地址匹配的行缓冲项。非相邻半行可合并；完整行直接 PutFull，不先 Get。未满尾行和压力淘汰仍交给原 line adapter 做带掩码 RMW。
- `EvuMemoryTagRouter.scala`：矩阵与 VMU 各保留本地 tag，共享默认 8 个全局 HBL2 source，支持多笔在途、乱序 Ack 及本地 tag 重号。矩阵不再固定占用 source 7；VMU 也可用 8 个本地 tag。
- `EvuTop.scala`：以全部缓冲、tag 和 Ack 排空作为矩阵完成条件，不以 `taskLast` 输入作为完成。保留矩阵 trace、物理 store trace、首个异常和 FP flags；按全局 tag 快照页映射。flush 期间保留已发送物理请求及 owner，排空后再取消 VMU 本地 tag，避免死锁或旧响应污染新任务。
- common-clock 模式下通过现有 PERF 输出 `evu_matrix_store_stall_cycles`、`evu_matrix_store_merges`、`evu_matrix_partial_writes`、`evu_memory_tag_stall_cycles`、`evu_memory_requests`、`evu_memory_responses`。需要启用 NORMAL 或更细的 perf print；没有新增 CSR 或 guest 指令。

默认整行覆盖情况下，12,288 个 32B 矩阵结果拍可合成 6,144 个 64B PutFull，载荷由原 RMW 路径约 1.5 MiB 降到 384 KiB。此为理想协议计数，不是实测周期收益；缓冲压力、尾行或非连续地址会保留部分 RMW。

这一批没有实现普通 VMU load 的行复用/跨请求 store 合并，也没有改 VRF bank、解除 `memoryLive`、增加算术单元或丢弃 PV 填充结果。VRF 与访存重叠的后续实现见下一节。不改 ISA 和 NEMU 语义，因此不要求仅为本轮重新编译 NEMU；服务器仍须使用支持先前 GatherImm 的匹配参考模型，并重建 RTL/emu 后再跑同一软件基线。

本地验证：`mill -i xiangshan.compile` 通过；下面 6 个 suite 的 21 项测试一次通过，随后新增同地址写入不得越过未确认 Ack 的测试，单独重跑 buffer suite 的 6 项也全部通过，共覆盖 22 项不同测试。行为测试使用 Treadle；顶层只生成内存中的 CHIRRTL，覆盖 VLEN=512/4096，不调用 firtool、Verilator 或 emu，也不编译 guest/native 二进制、不生成 Verilog。顶层展开仍有原 Saturn 路径的 enum cast/动态索引宽度警告；展开不等于全核动态验收。

```
PATH="$PWD/src/main/resources:$PATH" mill -i evuCheck.test.testOnly \
  evu.EvuMatrixWritebackSpec evu.EvuMatrixStoreBufferSpec \
  evu.EvuMemoryTagRouterSpec evu.EvuHbl2LineAdapterSpec \
  evu.EvuMatrixDequantSpec evu.EvuFpQuantSpec
```

组合测试覆盖非相邻行半片、相同 client-local tag、乱序 Ack、总线背压、跨行掩码尾部 guard、flush 后物理请求排空与重启。64 个常规 32B 结果在该测试中恰好产生 32 个 PutFull、0 个 Get；另外两个跨行尾片分别执行原 RMW。单模块测试还覆盖缓冲压力淘汰和首个访存异常。

服务器复测应保留同一份 `row_fused_v1` guest，先比较 `1,257,794` 周期基线及 QK/PV wait，再确认 GOOD TRAP、difftest `pending=0`、23 个样本及 `profile_errors=0`。物理 `checked_stores` 会因合并下降，矩阵结果拍仍应为 `12,288`、CUTE 写仍应为 `4,096`；不要把 store event 数减少本身当成丢结果。对齐完整行覆盖且无压力淘汰时，矩阵物理写预计少 6,144 次，但这不是所有访存的总数或精确周期预测。

矩阵写回批次服务器验收：2026-10-27

该批次日志仍为 `row_fused_v1`、cold、单次 profile，GOOD TRAP、`profile_errors=0`、`pending=0`。内核从 `1,257,794` 降到 `900,600` 周期，节省 `357,194`（28.40%，1.397 倍）。QK wait 从 `145,974` 降到 `9,932`，PV wait 从 `237,017` 降到 `16,024`，两项节省 `357,035` 周期，几乎解释全部收益。softmax/quant 为 `611,554`（67.90%），两类 output 合计仍为 `228,193`（25.34%）。

硬件计数 `matrix_store_merges=6144`、`matrix_partial_writes=0`、`memory_tag_stall_cycles=0`，请求和应答同为 `35056`。矩阵结果拍与 CUTE 写仍为 `12288/4096`。整次 host 时间 `1,301,289 ms`（21 分 41.289 秒），不是单独 kernel 时间。仍为 `fp64_checked=0`，不表示通过独立精度校验；与核内热跑 `322,302` 仍不是匹配 cold 对照。

本批 RTL：1RW bank 错位、读流水与单访存重叠

此批在上述 `900,600` 周期基线之后修改，已通过下文记录的 Flash Attention 服务器 emu/difftest；该工作负载结果不代替全功能回归。未改变测试内核、ISA 或 NEMU。

- 统一 `vrfBank(eg)`，默认 `VLEN=4096/DLEN=256` 时 `bank=chunk[2:0] XOR reg[2:0]`，SRAM row 仍为 `eg>>3`。数据读、mask 读、流水写、LL 写和未来写预约使用相同映射。每 lane 八个 bank、

每 bank `singlePort=true`，新增断言禁止同拍读写；容量仍为 16 KiB。v1/v2/v3 同号 chunk 可分散，v1/v9 等冲突仍仲裁，绝非增加端口。

- collector 支持消费当前响应同时发出下一 chunk 请求，并以单项缓冲留住预取返回。sequencer 仅对正常、等 EEW、非 reduction/permute/elementwise/mask-format 的后继 EG 发预取，保留 older-write 与当前 destination 检查；未获仲裁的预取走 demand 兜底。不会跨指令猜测下一组操作数，也不是无限深乱序读队列。
- `EvuCommandScoreboard` 保持命令级整寄存器 RAW/WAR/WAW、矩阵 scale/bias 读预约及单访存 owner。无依赖算术可在访存期间进入，新访存也不必等所有旧算术；第二条访存仍等前一条 completion 被接收，virtualBase 与异常状态仍只有一个 owner。
- 核内 `evuPreflightBusy` 只对未完成访存/控制命令及四拍 statusDrain 阻塞；普通算术不再单独阻塞页预检查。Launch 保持原全忙等待；decode 的 noSpec/blockBack、CSR drain、退休顺序与翻译取消路径未放宽。
- WAIT/FENCE 的活动寄存状态阻止 Top 接受后续命令，FENCE 仍等 VMU、矩阵、写缓冲、总线与完成预约排空。flush 禁止所有命令握手，清预约与预取；已发送请求保留 tag 待旧响应回收，取消的总线错误不写入新 owner，首错地址与 cause 保持同一快照。

common-clock PERF 新增计数（NORMAL 或更细输出）：

计数名	口径
`evu_vrf_bank_conflict_cycles`	至少一个 bank 存在不同 row 的读竞争且该 bank 不在写的周期数
`evu_vrf_write_blocks_read_cycles`	至少一个 bank 写操作挡住读请求的周期数
`evu_vrf_operand_wait_cycles`	至少一个 legacy operand request 尚未得到可消费返回的周期数
`evu_vrf_prefetches`	被物理读仲裁接受的预取操作数请求数
`evu_memory_order_stall_cycles`	输入访存被现有访存 owner 阻塞的周期数
`evu_register_hazard_stall_cycles`	访存顺序允许、但 RAW/WAR/WAW/矩阵读预约阻塞输入的周期数
`evu_memory_compute_overlap_cycles`	访存与非访存命令预约同时存活的周期数，不等于两单元实际同拍运算
`evu_preflight_serial_stall`	核内页预检查被访存/控制/statusDrain 阻塞的周期数

这些计数可相互重叠，不能相加当作 kernel 分解；预取会改变请求分布，bank 冲突次数也不能单独等同周期收益。采集同一次内核边界前后差值，或使用同 cold 工作负载对照，结合阶段周期判断。

本地已通过 4096 位 VRF 的全部 512 EG 写入和读回、三源并行、残余同 bank 冲突、写挡读、collector 连续请求/缓冲/reset 的 9 项测试；另 9 项覆盖生产 scoreboard、真实 VRF 多 chunk 连发/停顿/旧写依赖、WAIT/FENCE 及控制序列。AMU 定向测试覆盖普通算术放行、各类 memory/control 阻塞、statusDrain、redirect 和异常完成。完整 VMU+算术 Treadle 测试因解释运行过慢中止，保留为显式长测（`EVU_LONG_TREADLE=1`，默认 canceled），**未通过，不能算作全后端动态验证**。完整核 Scala 编译及 VLEN=512/4096 顶层内存 CHIRRTL 接线检查通过。这些检查不替代服务器完整 difftest。

收尾回归中，GatherImm、矩阵反量化、共享访存 tag 路由、同步 VRF 四个 suite 的 10 项测试全部通过；额外的纯映射检查覆盖每寄存器 1/2/4/8/16/32 个 chunk，验证 bank-row 映射的双射性。最后一次

`xiangshan.compile` 也通过。这里的 `VLEN=512` 仅为部分本地定向测试的缩小配置，每寄存器 64B，不是生产参数变更；生产仍为 `VLEN=4096`、`DLEN=256`、32 个寄存器共 16 KiB，每个物理 bank 仍为 1RW。4096 位配置中的“512 EG”则是整份 VRF 的 `32 × (4096/256)` 个数据组，不是 512 位 VLEN。

```
PATH="$PWD/src/main/resources:$PATH" mill -i evuCheck.test.testOnly \
  evu.EvuCommandScoreboardSpec evu.EvuSaturnOperandSpec \
  evu.EvuControlBarrierSpec evu.EvuControlSequencerSpec
PATH="$PWD/src/main/resources:$PATH" mill -i evuMmuCheck.test.testOnly \
  xiangshan.evucheck.EvuAmuCompletionSpec
PATH="$PWD/src/main/resources:$PATH" mill -i evuCheck.test.testOnly \
  evu.EvuGatherImmSpec evu.EvuMatrixDequantSpec \
  evu.EvuMemoryTagRouterSpec evu.EvuSynchronousVrfSpec
PATH="$PWD/src/main/resources:$PATH" mill -i evuCheck.test.testOnly \
  evu.EvuSkewedVrfSpec -- -z "every supported chunk count"
```

本地只允许 Scala 编译、Treadle 和内存 CHIRRTL 展开；不跑 emu、不编译 guest/native bin、不生成 Verilog。该批服务器验收使用同一 `flash_attention_evu_profile_cold` 软件和匹配现有 ISA 的 NEMU，对比基线为 `900,600` 周期；后续新优化应以本轮 `764,160` 周期作为同口径对照，并保留完成状态和停顿计数。

bank 错位、读预取及访存重叠批次服务器验收：2026-10-27

来源为本轮更新的 `flash_attention_break_cold_evu.log`。工作负载仍为 `q=128, kv=256, d=64`、`row_fused_v1`、cold、单次 profile，未运行 warm-up、FP64 参考或独立 output accuracy check。

阶段	上轮周期	本轮周期	减少周期	降幅
整个内核	900,600	764,160	136,440	15.15%
softmax + 量化	611,554	497,737	113,817	18.61%
output 累加	157,569	145,053	12,516	7.94%
output 累加 + 最终归一化	70,624	60,638	9,986	14.14%
QK/PV wait 合计	25,956	25,953	3	基本不变
PV pack	30,543	30,380	163	0.53%

内核加速约 `1.1785x`，83.4% 的净周期收益来自 softmax/量化。两类 output 合计为 `205,691` 周期；softmax 与 output 合计占本轮内核的 92.05%，仍是主要优化对象。内核退休指令仍为 `40,754`，访存请求/应答仍为 `35,056/35,056`，说明本批主要改善调度和等待，没有减少软件工作量。三个 RTL 改动同时启用，日志不能分别证明每一项的独立收益，需要消融对照。

服务器状态为 GOOD TRAP，`profile_errors=0`、`pending=0`、`efflags=0x1`；23 个阶段样本完整。difftest 检查 `19,456` 个 store、`11,948` 个 completion、`12,288` 个矩阵结果拍和 `4,096` 次 CUTE 写。`fp64_checked=0`，因此这是该工作负载的指令/事件一致性通过，不是独立输出精度验证，也不能代替 flush、异常等全功能回归。

整次仿真 host 时间为 `1,244,784 ms`，即 20 分 44.784 秒，较上轮缩短 4.34%。这不是 kernel 单独的 host 计时，不能按 kernel 周期降幅推算模拟器总耗时。

硬件计数	本轮值	当前解释
`evu_vrf_bank_conflict_cycles`	4,236	残余 bank 冲突规模较小
`evu_vrf_write_blocks_read_cycles`	1,792	继续只优化这一项的空间有限
`evu_vrf_operand_wait_cycles`	41,868	仍有供数等待，不能全部归因于 bank 冲突
`evu_vrf_prefetches`	47,744	预取路径实际工作，但不等于节省同样多周期
`evu_register_hazard_stall_cycles`	107,823	整寄存器依赖预约值得优先细分检查
`evu_memory_compute_overlap_cycles`	5,337	当前软件暴露的预约重叠有限，不等于净节省周期
`evu_memory_order_stall_cycles`	0	本轮没有观察到该入口顺序条件的阻塞
`evu_memory_tag_stall_cycles`	0	本轮没有证据支持优先增加 tag 数量
`evu_preflight_serial_stall`	4	核内页预检查的该阻塞条件当前占比较低

以上均是全程序末尾快照，可能包含初始化和收尾，不是只在 kernel 边界采样的互斥周期分解；计数相互重叠，不能相加。依赖阻塞含真实依赖，不能把 `107,823` 全部视为可消除周期。矩阵合并仍为 `6,144`，partial write 为 0，QK/PV wait 已仅占内核约 3.40%。

与历史核内热跑 `322,302` 周期相比，本轮周期为其 2.37 倍，还需减少 `441,858` 周期，即本轮的 57.82%。cold/warm 和插桩边界尚未匹配，该比值只是距离历史目标的参考，不是严格匹配的最终加速比。继续微调 bank 仲裁不足以解释这样大的差距。

基于 764,160 周期的下一步方向

以下记录本轮结果分析提出的方向，尚未实现或实测收益。

软件方面：

1. 压紧 `p_scale` 和 `row_sum`，将比例乘法、分母保护、比例保存、行和更新安排进已有 VL=1 区间，减少 64 份重复计算和搬运。比例数组可直接供 PV 使用，删除 `packed_p_scale` 的逐行复制。两类状态容量分别可从 64 KiB/32 KiB 降到 1 KiB/512 B；容量变化不直接等于总线或周期变化。
2. 使用两份 PV 结果缓冲，将同一 key tile 两个 chunk 的 output 更新合成一遍，保持 `(旧输出 + PV0) + PV1` 的 FP32 加法顺序，省去两份 chunk 之间的 output 写回和重载。默认形状可减少约 128 KiB 逻辑 output 读写，代价是额外缓冲。保留首次与零相加、第二 tile 的 alpha 缩放、末次归一化的运算语义。
3. 按两行/四行交错组织独立临时寄存器，隐藏部分依赖和量化等待，并评估分批比例和倒数计算。各行保持独立归约，不能直接将普通 VL 扩大后跨行求和；需核对 32 个寄存器的生命周期及当前单 FP sequencer 的限制。
4. 复用 QK chunk 最大值，评估用 `exp2(chunk_max-row_max)` 替代概率最大值归约。必须检查近似 exp2 的单调性、舍入、量化边界及新增工作的成本；若不能保证数值行为，则考虑在硬件生成概率时同时统计 max，避免再扫描。

RTL 方面：

1. 优先评估普通 Saturn 命令之间恢复内部 EG 级 chaining。当前外层整寄存器预约一直保留到完成，可能遮蔽已有细粒度依赖能力；跨量化、矩阵参数等路径先保守保护。保留单访存 owner、异常、flush 和 WAIT/FENCE，增加 RAW/WAR/WAW 及普通/特殊路径分类计数，不能直接删除依赖检查。

2. 增加指令内单元素操作、向量消费标量元素、量化内部复用单个比例的能力。当前 640 次有效除法伴随 1,280 次逐行 VL 切换，EVCFG 访问又要求 EVU 排空；减少这些切换比单纯增加除法器更值得优先验证。新指令语义需同步 NEMU，不能仅取消 CSR drain。
3. 优化普通 VMU 的 32B 到 64B 访问放大：合并同一缓存行的在途 load 并共享返回，在单条 store 命令内合并完整行。尾部和命令结束强制排空，不能等下一条 store 补齐而与单访存 owner 形成死锁；保留错误、flush、tag 和读后写可见性。
4. 推进矩阵结果流与消费者融合：QK 转换时统计 chunk max，PV 结果进入输出累加缓冲，最终归一化后再写出，减少 `CUTE -> HBL2 -> EVU -> HBL2` 往返。有效 output 为 32 KiB，不能整份放进现有 16 KiB VRF；需按块处理或增加独立分 bank 输出 SRAM，现有 VRF 每个 bank 保持 1RW。
5. 数据供给和依赖改善后，再评估 exp、加乘、归约的发射上下文并行及计算单元扩展。现有归约已有四路部分累加，不能当作完全逐元素串行；先增加 sequencer 发射率、归约等待及 exp/FMA 利用率计数，避免盲目堆 FMA 或除法器。

建议下一批先评估“紧凑比例/行和 + 普通命令 EG 级 chaining”，再做指令内标量支持与普通 VMU 缓存行复用，随后推进矩阵/softmax/output 流式融合。可将 softmax 不超过 18 万、output 不超过 6 万、其他阶段不超过 6 万、总计不超过 30 万周期作为设计验收预算；这是目标分配，不是现阶段可保证的性能预测，各项收益不能直接相加。

已落实的软件子集：compact_batched_v2

本批仅修改软件及其构建/计时文档，未修改 RTL、ISA 或 NEMU。`flash_attention_evu_profile_cold.c` 仍通过 profile/breakdown 引入共享内核，默认只跑一遍 Q128/KV256/D64，无预热、baseline 或 FP64 对比；输入初始化、V 整理和完整算子工作量保留。

- `p_scale[2][Q]` 为 1 KiB，`row_sum[Q]` 为 512 B，取代原 64 KiB/32 KiB 的广播数组；比例、行和仅在 VL=1 下计算/存储。删除 `packed_p_scale` 及其逐行复制。`row_max` 和 alpha 的广播表示暂未改变，没有实现 chunk-max 替代归约。
- 默认使用两份各 64 KiB 的 PV 结果缓冲；矩阵任务仍顺序执行并等待结果可见，每个 key tile 的两个 PV 在同一 output 更新中依次相加后只写一次。保留首次正零加法、第二 tile 的 alpha 缩放，以及原两次加法的先后与操作数顺序；不改成 FMA。按源码计数减少 128 KiB 逻辑 output 读写，不承诺同等物理流量或周期降幅。
- 四行各有独立 P0/P1、max、alpha/比例、sum 寄存器；两份概率生成后复用 max，alpha 用于行和更新后复用为倒比例。共享 VL=1 窗口，将比例相关 VL 切换由 1024 次降为 256 次，归一化相关由 256 次降为 64 次，仍为 640 个有效除法元素。各行独立归约，VL=64 的量化约束保持不变，没有新增硬件并行归约上下文。
- 默认日志为 `kernel=compact_batched_v2 profile_schema=compact_pv_pair_v2 row_batch=4 compact_state=1 pv_pair=1 output_passes=2 scale_vl_switches=256 normalize_vl_switches=64`，样本为 21，普通 output 与 normalized output 各 1 次。原 `row_fused_v1` 为 23 样本，比较时应合计两类 output，而非直接比较每次平均周期。

Makefile 默认 `EVU_FLASH_ROW_BATCH=4 EVU_FLASH_MERGE_PV=1`，cold 目标和默认 bin 文件名不变。可选行批量 1/2/4、PV 合并 0/1，用于后续消融；不同配置使用独立对象目录，非默认镜像名带 `rowsN-pvM`，不会在一次 cold 运行中执行多份配置。本软件批次只需在服务器重新编译默认 cold guest，可沿用通过 764,160 周期测试的 emu/NEMU。

本地 RISC-V GCC/Clang 对 4 个入口、6 种配置的无产物语法检查通过。临时 Clang AST 检查器将实际 C 中的寄存器操作与 `row_fused_v1` 的有序 FP32 表达式图比较，6 种配置各连续运行两次，检查未定义状态读取、比例、量化表达式、行和、输出和切换次数，通过。矩阵/归约为不透明确定性节点，不执行 NEMU/RTL、不计算

硬件 exp2，也不构成服务器数值或周期验收；未编译 bin/目标文件或生成 Verilog。服务器验收口径为默认 cold 日志的 GOOD TRAP、difftest pending=0、21 样本和 profile_errors=0。后续硬件收益应与最新 compact_batched_v2 结果对照，不继续把旧软件的 764,160 周期作为同软件基线。

下一轮五批 RTL 优化方案

以下五批是后续设计方案，尚未实现；与前文已经完成的 bank 错位、读预取、单访存重叠及矩阵写合并分开。目标是在保持每个物理 VRF bank 为 1RW 的前提下，提高已有计算资源的利用率，并减少普通访存及标量状态处理的开销。所有路径继续保留异常、flush、WAIT/FENCE、完成事件和 difftest 语义。

第 1 批：跟踪真实写回，再恢复普通命令 EG 级 chaining

当前外层 `EvuCommandScoreboard` 对普通 Saturn、特殊量化和矩阵参数访问统一保留整寄存器预约，直到 completion 被接收。Saturn 内部已有 EG 级 RAW/WAR/WAW 跟踪，但部分可重叠的工作被外层提前拦住。

不能直接删除外层检查。当前 `LoadSequencer` 可在数据交给 VRF 的 LL 写缓冲时推进 EG；`RegisterFile.scala` 的 `llPending` 可能因同 bank 流水写而延后提交到 SRAM。只依据 sequencer 推进就释放依赖，会让消费者读到旧值，或让后写指令越过尚未提交的旧写。

实现步骤：

1. 为预约增加普通 Saturn、特殊路径等类别，只对已证明由内部依赖系统完整覆盖的普通命令对恢复 chaining。先开放普通算术之间，再开放 load/compute；量化和矩阵参数读继续保守保护。
2. 为 LL 缓冲中的写请求携带 VAT/owner、EG 和有效掩码，并导出待写依赖及实际 SRAM commit。将这些信息纳入内部 RAW/WAR/WAW、VAT 生命周期和完成条件；最后一个 EG 进入缓冲不等于指令已经写完。可先保留整个待写 EG 的保守依赖，完成单元素语义后再评估细化。
3. 保留单访存 owner，store 完成仍须等物理 Ack，不能为开放 load chaining 提前释放访存状态。flush 清理未提交内部写及其依赖；已发送总线事务继续按现有规则排空，避免 VAT/epoch 重用后被旧返回污染。
4. 增加普通/特殊路径阻塞分类、pending write 等待、实际 EG chaining 活动计数；现有 register-hazard 周期包含真实依赖，不能全当作可节省周期。

主要修改：`evu/EvuCommandScoreboard.scala`、`EvuVectorBackend.scala`、`backend/{Backend,LoadSequencer,ExecuteSequencer,RegisterFile,RegisterAccess}.scala` 和 `common/Bundles.scala`；需要时补接 frontend 的 VAT 释放/依赖接口。这里及下文的 `evu/` 路径均相对 `src/main/scala/xiangshan/`。

验收应覆盖相同/不同 EG 的 RAW/WAR/WAW、掩码写、load 最后一拍被 1RW bank 写阻塞、完成口背压、VAT 重用、flush 和普通/特殊命令交叉。每个 bank 同拍至多一次读或写，绝不以增加 SRAM 端口解决冲突。保持 ISA 和运算顺序时不要求修改 NEMU 的指令语义。

第 2 批：普通 VMU 的 64B 缓存行合并

矩阵写合并已经实现，这一批针对普通 VMU。当前 `EvuHbl2LineAdapter` 按逻辑请求/tag 独立处理；相邻两个 32B load 可各发一次 64B Get，两个 32B store 则可能分别进行 Get/merge/Put。

- load：在同一命令范围内，按物理缓存行地址匹配在途请求，一个 Get 的返回按 offset 分发给多个逻辑 owner。响应必须保留每个 owner 的 tag、错误和背压状态。可用短期行缓冲覆盖后到的相邻半行，但不跨命令保留未经一致性保证的读缓存。

- store：在同一命令内收集 data/mask，完整 64B 行直接 PutFull；部分行仍走现有 RMW。每个逻辑 store owner 都在对应物理写 Ack 后才应答，不能在收进缓冲时就报完成。
- 新增明确的命令标识和请求发出结束通知，不能等待命令 completion 才排空尾行，否则 completion 本身又在等尾行 Ack。命令末尾、缓冲压力和必要的顺序边界都触发部分行排空；不能等下一条命令补齐缓存行。
- 保留跨行请求拆分、同地址读写顺序、页检查、首错、flush 与 tag 回收。物理请求被合并后，全局 source 与多个逻辑 tag 的生命周期需要一起调整。

主要修改：`evu/EvuHbl2LineAdapter.scala`、`EvuMemoryTagRouter.scala`、`EvuTop.scala`、`EvuVectorBackend.scala` 及 VMU 请求接口；命令边界与 owner 元数据必须从生产者完整送到合并层。

对齐、完整、连续的 256B 向量，理想情况下 load 从 8 个 Get 变为 4 个 Get；store 从 8 个 Get + 8 个 Put 变为 4 个 Put。小标量访问、尾部和压力淘汰不能套用这一比例，也不承诺相同比例的 kernel 周期下降。除完成粒度或 trace 接口确实变化外，不改变 guest/NEMU 指令语义；difftest 必须继续校验物理有效字节与完整逻辑效果。

第 3 批：拆分 FP 发射上下文，再增加独立归约上下文

目前单个 FP sequencer 管理多类计算，某条长指令占用发射状态时，其他独立行的计算也可能无法前进。应先让已有 FMA、比较、exp2 等资源能由独立指令上下文持续供数，再评估是否需要增加计算单元。

1. 增加明确的 FP issue path 分类，例如 exp、加乘/比较和归约；按 opcode 指定支持集合，避免同一功能单元被意外复制或一条指令被多次接收。现有 `MultiFMA` 配置会增加 FMA 路径，并不自动等于复用已有资源的发射拆分。
2. 对共享功能单元做带 VAT/owner 的仲裁和返回路由，保留每条指令的 flags、尾拍、写预约及完成信息。并行路径的 VRF 读写仍服从现有 1RW bank 仲裁，资源争用时背压。
3. 当前 `backend/SpecialSequencer.scala` 的四个 accumulator slot 是一条归约指令的部分累加器，不是四条行归约上下文。多行并行需要独立保存各自的四个 slot、初始化、busy、merge/fold 状态，并给返回增加 context 标识。
4. `backend/Backend.scala` 当前断言同拍最多一个 accumulator writeback；多上下文后须设计能背压或缓冲的回写仲裁，不能仅删断言。每条归约保留原 slot 分配和合并顺序，避免 FP32 加法重排导致结果改变。

主要修改：`evu/common/{Parameters,Bundles}.scala`、`backend/{Backend,ExecuteSequencer,SpecialSequencer,IssueQueue,RegisterFile}.scala`，以及实际需要携带 context/背压的 `exu/fp/` 接口。定向验证独立指令并行、同 bank 写竞争、归约返回交错、flags 归属和 flush。保持原指令结果及归约次序时无须新增 NEMU ISA；若改变归约算法或舍入顺序，必须单独说明并同步参考语义，不能作为纯调度优化混入。

第 4 批：指令内单元素计算、向量消费标量及标量量化比例

软件四行分批已经将 VL 切换从 1,280 次降为 320 次，仍有 512 次量化比例除法和 128 次归一化倒数。下一步让指令自身描述有效元素和标量源，避免为每个标量窗口全局切换 VL/排空 EVU。

- 单元素操作：提供只处理元素 0 的 FP32 load/store、加减乘、max、div 和所需 exp2；写寄存器时保留其他元素，访存只访问实际标量字节，不越界读写邻居。
- 向量与标量操作：从选定源寄存器的元素 0 读取一次，在执行端广播，供整行减 max、乘 alpha 或倒数；不再要求先把标量写成完整比例向量。明确 `vs2 op vs1`、非交换运算及源目标重叠语义。

- 量化：让 FpQuantToCute 支持读取一个标量倒比例，并在内部复用，保留旧的向量比例模式以兼容现有 guest。量化舍入、饱和与 CUTE 写入顺序保持原语义。
- 先固定编码、SEW、VL=0/vstart、掩码、保留元素、异常和 flags 规则；不新增全局 CSR drain 绕过，也不通过临时改写全局 VL 实现局部操作。标量捕获和待写也要纳入第 1 批的依赖跟踪。

RTL 主要修改 `evu/{EvuIsa,EvuParams,EvuVectorBackend,EvuSpecialSequencer,EvuFpQuant}.scala`、`backend/decode/EvuDecoder.scala`，以及必要的 Saturn decode/微操作/操作数接口。软件修改 `software/evu/include/evu_intrinsics.h` 和共享内核 `flash_attention_evu_test.c`，让 cold 的 include 链自动使用新路径；在 breakdown 中打印新配置，旧 guest 路径保留作对照。

这一批需要同步 NEMU：本地仓库为 `NEMU`，服务器使用的仓库名可能为 `NEMU_evu`。主要涉及 `src/isa/riscv64/instr/evu/{evu-internal.h,compute.c,model.c}`，按实际解码入口补充新编码识别，并验证输出有效掩码、保留字节与完成事件。采用已有 VRF 元素 0 时不必新增架构状态；仅在最终设计增加新状态或导出字段时，才扩展 `evu-state.h` 和 difftest 接口。服务器必须同步重建 NEMU、emu 和 cold guest。

这批可消除剩余标量窗口切换并减少显式广播/重复元素工作，但不把 640 个必要除法直接降为零。无需优先增加除法器；收益需要结合 CSR 等待、FP 发射和供数计数实测。

第 5 批：QK、softmax 与 PV 消费的流式融合

目标是把数据留在 EVU 的行块工作集内，减少矩阵结果先落 HBL2、再被 EVU 读回的往返。

1. QK 结果转换/缩放时生成按行、按 chunk 的最大值，结果进入行块缓冲；后续软最大值及量化直接消费。若用 `exp2(chunk_max-row_max)` 替代概率最大值归约，先证明硬件近似 exp2、舍入和量化边界满足所需语义，否则在生成概率时同步统计 max。
2. P 保留直写 CUTE；PV 反量化结果直接进入局部 output 累加器，按原顺序执行 alpha 缩放、PV0/PV1 加法和最终倒数乘法，最后写出有效输出。保留独立 FP32 舍入，不静默换为 FMA。
3. 完整有效 output 为 32 KiB，大于现有 16 KiB VRF，不能宣称直接全量驻留。选择分块处理，或增加独立分 bank 的 output SRAM；后者也需明确容量、端口、仲裁和背压，不改变现有 VRF bank 的 1RW 属性。
4. 固定任务描述符、row/chunk/context 标签、矩阵消费者握手、异常及完成边界，防止 CUTE 和 EVU 互等。新增计算/任务语义时同步 NEMU、矩阵参考模型和 difftest，并增加流式路径专用 profile schema。

主要涉及 `evu/{EvuTop,EvuSpecialSequencer,EvuMatrixDequant,EvuMatrixStoreBuffer,EvuFpQuant}.scala`、新增行块/输出缓冲、任务描述符及共享软件内核。保留 `compact_batched_v2` 的访存路径用于回归；流式 PV 路径可能替代现有两份 PV 内存缓冲，不能不加选择地叠加两套搬运方案。

集成交付顺序与仅实施第 1–4 批的范围

可以一次交付五批，但应按共同接口先定稿、内部逐批验证、最终统一服务器验收的方式推进。先固定 owner/VAT/epoch、实际写提交、命令末尾通知、归约 context 和标量指令语义，再做实现，避免各批对“完成”的定义不同。第 1 批建立正确依赖边界；第 2 批减少普通访存放大；第 3 批利用软件已经暴露的多行工作；第 4 批让标量操作不再全局排空；第 5 批进一步重构数据流。各项周期收益相互影响，不能直接相加。

如果先做第 1–4 批，范围是 `XSAI_external_vector` 的后端、访存适配和指令/软件封装，加上 `NEMU` 的新增指令参考实现与对应定向测试。保留现有矩阵任务、QK/PV 内存结果缓冲和四行软件调度，不实现第 5 批的专用流式 attention 任务或 output SRAM。第 1–3 批应先用现有 cold guest 回归，第 4 批完成后再切换支持标量指令的新 guest。

为各批保留独立配置开关和旧路径，便于归因与回退。一次服务器 cold 执行只测所选组合，可得到整体收益；分开衡量四批收益仍需消融运行，单份总计日志不能给出四个独立加速比。性能入口继续只运行一遍完整 Q128/KV256/D64，不加预热、baseline 或 FP64；指令正确性通过定向边界测试与匹配 NEMU 的 difftest 验证。

本地检查继续限于无产物语法检查、Scala 编译、Treadle 和内存 CHIRRTL，不运行 emu、不编译 guest/native bin、不生成 Verilog。服务器最终检查 GOOD TRAP、pending=0、阶段样本完整、flags/异常及逻辑输出一致，并与相同软件、相同冷/热条件和计时边界的基线比较。超过历史核内周期是设计目标，需另做匹配条件对照，不能预先保证第 1–4 批已经足够。

测试算法优化方法

以下各条保留最初路线分析及历史表示，最新已实现范围以此前的 `compact_batched_v2` 小节为准。

1. 两个版本实际做了什么

核内测试入口：

- `nexus-am/tests/exp2test/tests/performance/vfexp2_flash_attention_ame_qk_lmul8_compare.c`

核内主要实现：

- `nexus-am/tests/exp2test/vendor/xsai_blas_rvv/flash_attention_ame_vector_max_optimized.c:255`
- `nexus-am/tests/exp2test/vendor/xsai_blas_rvv/softmax_exp_sum_quantize_4row_m4.c:85`

EVU 实现：`XSAI_external_vector/software/evu/flash_attention_evu_test.c`，普通测试与 breakdown 模式共享内核。

阶段	不带 EVU 的核内实现	当前 EVU 实现
QK 后处理	CUTE 输出 INT32；RVV 转 FP32、乘比例时，顺便计算并保存 chunk 最大值	CUTE 经 EVU 转 FP32 写回；EVU 再读 scores 计算最大值
在线状态	每行各保存一个 max、sum、alpha；alpha 跨 4 行一起计算	每个行状态保存 64 份副本，按行反复读写
Softmax	交错处理 4 行，使用向量与标量运算；从 chunk 最大值计算量化比例	单行处理，读取广播状态；对指数结果再次归约最大值
P 量化	RVV 量化、窄化、写内存，再加载进 CUTE	已经直接量化写入 CUTE，这是 EVU 当前的优势

阶段	不带 EVU 的核内实现	当前 EVU 实现
PV 更新	CUTE 输出 INT32，软件反量化并累加到输出	EVU 反量化后先写 pv，再读 pv 和旧输出，相加后写回
最终归一化	每行一次倒数，再乘整行	除法优化后已采用相同思路，但仍有配置切换和广播

2. 紧凑行状态与多行批量计算

首先把行状态从“64 份副本”改成紧凑存储，并跨行批量计算。

`row_max`、`row_sum`、`alpha`、`p_scale` 本质上都是每行一个数。当前这些数组占 160 KiB，紧凑表示只需要 2.5 KiB。按当前 C 程序的访存语义统计，这些状态在一次内核中产生约 704 KiB 的逻辑读写；同样访问次数下，紧凑数据只有约 11 KiB。

这不等于物理总线流量直接下降 64 倍，还需要批量访问和硬件标量操作支持，但说明这里存在大量可消除的数据搬运。

这里还浪费了计算：第二个 key tile 的 alpha，每行实际只需要一次 exp2，当前却对 64 个相同元素计算，合计 8,192 个元素运算代替了 128 个有效值。应把多行状态装进一个向量，批量更新 max、alpha、sum，而不是把同一行状态铺满一个向量。

3. 保存并复用 chunk 最大值

避免 softmax 再归约一次最大值。

核内版本在 QK 后处理时保存 `chunk_max`，随后计算：

```
max_exp = exp2(chunk_max - row_max)
```

当前 EVU 在算出整行概率后，又执行 `XV_FREDUCE_MAX`；默认形状共执行 512 次这样的归约。可以借鉴核内做法，将 chunk 最大值和行最大值在同一次 scores 扫描中得到并保留。

采用这个变化时，要验证近似 exp2 的数值行为和量化结果，不能只依据实数恒等式认为它们逐位相同。

4. 按多行组织工作与 VRF 驻留

核内采用 4 行交错，目的是暴露独立工作、隐藏指令依赖。EVU 可以按 4、8 或 16 行组织 scores、概率和行状态，减少反复装载，并批量计算比例与倒数。

不能仅把 VL 调大后跨行做一次普通归约，那会把不同的行混在一起；必须保留独立归约上下文，或提供分段归约。

整份输出是 32 KiB，也放不进当前 16 KiB VRF，必须按行块安排驻留。

5. 减少输出初始化、读写和最终扫描

第一份 PV 贡献可以直接初始化输出，省去“先清零，再读零、相加、写回”。最后一次 PV 累加完成时，行和已经可用，可以在同一行处理中接着归一化，省去之后再加载、写回整份输出。

这些重排应尽量保持原有加法、乘法的舍入顺序，不必先改变算法精度。删除第一次与零相加时，仍需验证 signed zero 等边界行为，不能不经验证就声称所有输入逐位等价。

6. 减少逐行 VL 切换，保持比较口径

比例和归一化已改为每次只算一个有效标量，默认形状每内核为 512 次比例除法加 128 次归一化倒数除法，而非每条向量的 64 个重复除法。但为此引入了 1,280 次 VL 配置切换，应尽量批量计算比例和倒数；进一步消除这些切换则需要 RTL 支持。

此前的广播优化已将五处广播改成 `XV_BROADCAST` / `GatherImm`，减少广播执行微操作，但仍不等于紧凑行状态或完整的向量-标量操作支持，也不能直接把微操作下降比例视为整个内核加速比。

删除 FP64 参考只缩短整个仿真任务的耗时，不会继续降低已经排除参考计算的内核周期。performance-only 模式适合周期定位，算法改变的正确性仍需另行验证。

另外，两份实现都使用固定的 128×128 CUTE tile，不能把填充计算全部视为 EVU 独有开销。

硬件 RTL 优化方法

1. 资源比较与执行结构

VRF 大很多，不代表每拍算力也大同样的倍数。

EVU 的 `VLEN=4096`、`DLEN=256`，VRF 容量为 16 KiB；但当前只有一个 FP 执行 sequencer，FMA、加减乘、比较、指数等共享这条发射路径。

核内配置有两条 128 位向量 FMA 路径，聚合宽度同样是 256 位，见 `XSAI/src/main/scala/xiangshan/Parameters.scala:473`。EVU 的指数通路更宽，但现有配置并不意味着所有 FP 运算都拥有明显更高的聚合峰值。

2. 消除 32 字节访问到 64 字节缓存行的放大

目前 MLEN=32 字节，而 HBL2 访问单位是 64 字节。普通 32 字节加载会读取整行；32 字节存储则通常执行：

```
Get 64B -> 合并 -> PutFull 64B
```

相邻两次半行存储因此可能重复读写同一行，见 `XSAI_external_vector/src/main/scala/xiangshan/evu/EvuHbl2LineAdapter.scala:77`。

应增加加载缓存行复用和存储合并：两份半行数据收齐后直接写完整缓存行；不完整尾部再走原来的处理。实现时需要保留读后写可见性、应答语义和 fence 排序，不能仅延迟写回而不处理依赖。

3. 矩阵输出持续接收与多笔并发写回

目前矩阵输出端只保存一拍结果，并等待该拍的访存应答后才接收下一拍；它还固定使用一个保留 tag，见 `XSAI_external_vector/src/main/scala/xiangshan/evu/EvuTop.scala:278`。

历史运行有 12,288 个矩阵结果拍，数据量 384 KiB；按当前半行 RMW 路径，仅这些结果就对应约 1.5 MiB 的 HBL2 读写载荷。这是协议路径推导，不是实测 DRAM 总线带宽。

应增加结果 FIFO、缓存行合并和多个写回 tag，让反量化、接收下一拍、上一拍写回应答能够重叠。PV 中无用的填充特征还应尽早丢弃，减少后处理与写回。

4. 解除访存与计算间过粗的串行限制

当前外层有明确条件，见 `XSAI_external_vector/src/main/scala/xiangshan/evu/EvuVectorBackend.scala:269`：

```
!memoryLive && (!isMemory || !hazardValid.asUInt.orR)
```

它意味着访存执行期间后续计算不能进入，而新访存又要等待先前命令完成。此外，核内访存预检查也等待 EVU 不忙。

因此不能只删除 `memoryLive`：需要一起设计访存排序、寄存器依赖和预检查状态，使无依赖加载、计算、存储能够重叠，只有真正相依或遇到 fence 时才等待。还需覆盖异常、flush、tag 回收与完成通知，避免以正确性换取表面吞吐。

5. 修复 VRF 同 bank 冲突，再提高读取吞吐

当前：

```
eg = 寄存器号 × 16 + chunk
bank = eg 的低 3 位
```

所以 v1 和 v2 的同号 chunk 总落在同一 bank。双源运算无法同时读两个操作数；1RW bank 遇到写回还会阻塞读取，见 `XSAI_external_vector/src/main/scala/xiangshan/evu/backend/RegisterFile.scala:189`。

优先考虑让 bank 映射包含寄存器编号的错位布局，并同步修改读地址、写地址和写预约，不能只替换一个 bank 计算函数。

其次，现有读 collector 在等待返回时不能接收下一笔请求，应增加流水请求和操作数缓冲。本次“返回直通”只去掉了额外等待的一拍，没有解决这些吞吐限制，也没有增加 VRF 读写端口。

6. 标量操作与更细的依赖跟踪

需要支持“向量与某个标量元素运算”、标量倒数/比例计算，以及量化直接消费一个标量比例。这样能减少显式广播、比例向量读写和 VL 切换。

当前外层普通命令还按整个寄存器保留依赖，会限制 Saturn 内部已有的分组 chaining；应先细化普通命令之间的依赖，跨量化和矩阵路径再保留必要的保护，而不是所有路径都等待整条指令结束。

7. 扩展 P 直写优势，再考虑增加计算单元

目标应是：

- QK 结果到达时完成缩放与最大值统计。

- PV 结果到达时直接累加到输出缓冲。
- 尽量减少 `CUTE -> HBL2 -> EVU -> HBL2` 往返。

矩阵结果可以进入按行组织的缓冲或 VRF，而不是要求整份 score tile 全部驻留。

等这些路径能持续供数后，再拆分 FMA、指数、归约的发射上下文，评估增加第二条 FP 路径。除法已经降至每内核 640 个元素，现阶段堆除法器的优先级较低。

8. 推进顺序与评估边界

建议依次推进：

1. 紧凑行状态与重复工作消除。
2. 缓存行合并及矩阵并发写回。
3. 解除访存串行化。
4. VRF bank 错位与读流水。
5. 增加计算并发。

历史 division-only breakdown 中，softmax 和 max/alpha 合计占 62.4%，但即使把这两项完全消除，仍剩 740,602 周期。所以要超过核内的 322,302 周期，算法的数据复用和硬件的数据输送必须一起改，单独优化某个算术单元不够。

这些阶段计时包含阶段内等待，不能直接归因为某个硬件单元的独占耗时。每轮应使用相同输入形状、精度与量化规则、冷/热条件、计时边界，记录 kernel 周期、阶段周期、完成状态及必要的硬件停顿计数，再更新结论。